Artificial Models for Multimodal Creativity
Winter semester 2026/27 · Sound, symbols, sensors

Learning Sequences

LESSON 3 · MARKOV CHAINS, RECURRENT NETWORKS, SELF-ORGANISING MAPS

Prof. Dr. Alessandro Anatrini

Hochschule für Musik und Theater Hamburg
Conservatorio di Musica “G. Nicolini”, Piacenza
Accademia di Belle Arti di Brera, Milano

Today

1	Music as a sequence of numbers; Markov chains, from Markov's count of letters to `ml.markov` in Max	45 min
2	Recurrent networks, training on the next event, temperature; Performance RNN and the Transformer	55 min
3	Self-organising maps: a map of harmonies, a map of the settings of a plugin	25 min
4	Exercise 3: a Markov chain on your own MIDI files, Performance RNN steered from Max, two maps to play	85 min
5	Training your own model; discussion: whose performances are in the data, and when a model copies	20 min

From following to generating

EXERCISE 2

The hidden Markov model of part C followed a gesture while it happened: at every frame it estimated which recorded gesture was being performed and how far through it you were.

The model only answered questions about a sequence that you produced.

EXERCISE 3

A model learns from a corpus of MIDI files which event usually follows which, and then produces the sequence itself: it chooses the next note, appends it, and chooses again.

You no longer play every note. You choose the corpus, and you steer the generation while it runs.

In the table of lesson 1 this is self-supervised learning: the target of each example, the next event, is taken from the data itself, and no one has to label anything.

Sequences, 1913–2026

	COUNTING TRANSITIONS 1913–1985	RECURRENT NETWORKS 1986–2016	ATTENTION 2017–2022	PRETRAINED MODELS 2023–2026
Models of sequences	**1913** Markov counts vowels and consonants in 20 000 letters of *Eugene Onegin* **1948** Shannon generates English text from letter and word statistics **1966** Baum and Petrie: hidden Markov models	**1990** Elman: a network with a recurrent hidden layer **1991–94** Hochreiter; Bengio et al.: the gradient vanishes over long sequences **1997** LSTM (Hochreiter, Schmidhuber) **2014** GRU (Cho et al.)	**2017** the Transformer **2018** GPT: a Transformer pretrained on the next token	**2023–26** language models of up to trillions of parameters; the same next-token training
Music	**1956** Pinkerton: tunes from the transitions of 39 nursery songs **1957** Hiller and Isaacson, *Illiac Suite*, experiment 4; Brooks et al.: hymn tunes, orders 1 to 8 **1958–59** Xenakis, *Analogique A* and *B*	**1989** Todd: a recurrent network writes melodies **2002** Eck and Schmidhuber: an LSTM plays blues **2003** Pachet, *Continuator*: continues a player's phrases **2006** OMax (IRCAM): improvises from what it has just heard **2016** folk-rnn: 23 000 Irish tunes	**2017** Performance RNN; DeepBach **2018** Music Transformer **2019** MuseNet; the Bach Google Doodle **2022** Notochord: MIDI in under 10 ms	**2023** Anticipatory Music Transformer: accompaniment and infilling **2025** Aria, pretrained on transcribed piano recordings; Aria-Duet plays with a pianist on a Disklavier **2026** YuE2 writes melody and chords as a score before the audio of a song

Notes as numbers

Pitch	Velocity	Start, s	End, s
48 C3	56	0.957	10.829
36 C2	52	0.970	10.820
72 C5	61	1.617	1.983
74 D5	66	1.921	2.225
76 E5	71	2.164	2.451
67 G4	61	2.426	2.724
76 E5	66	2.681	2.958
65 F4	63	2.935	3.215

The first notes of a performance of the Prelude in C major from book II of Bach's *Well-Tempered Clavier*, as recorded by a Disklavier at the International Piano-e-Competition and stored in MAESTRO.

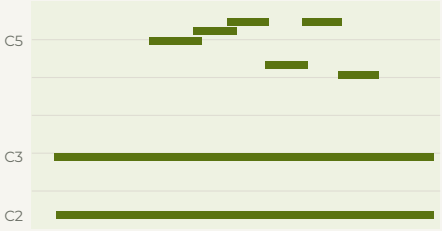
A MIDI file stores events: a key goes down (note-on), a key comes up (note-off), the pedal moves. Read as notes, each note is four numbers. The performance lasts 240 seconds and contains 1925 notes: a table of shape (1925, 4).

The two low Cs are held for ten seconds while the right hand plays. Notes overlap: a piano performance is not one line of pitches but several at once.

MAESTRO v3.0.0, 2014, `MIDI-UNPROCESSED_16-18_R1_2014_MID--AUDIO_16_R1_2014_wav--1`. MIDI 60 is C4.

Two ways to write the same music

PIANO ROLL



time, in steps of 10 ms; one row per pitch

SEQUENCE OF EVENTS

960 ms	53–56	on C3	10 ms	49–52	on C2	650 ms
61–64	on C5	300 ms	65–68	on D5	60 ms	off C5

time shift · velocity · note-on · note-off

A **piano roll** is a grid: one row per pitch, one column per time step, like a spectrogram of the score. It has a fixed size and is suited to convolutional networks, but most of its cells are empty.

A **sequence of events** reads like the MIDI file itself: time passes, a loudness is set, a key goes down. Every event is one symbol from a fixed vocabulary, as a word is in a text. This is the form used by Performance RNN and by most current models of music.

A vocabulary of 388 events

Event	Count	Meaning
note-on	128	a key is pressed, MIDI pitch 0–127
note-off	128	a key is released
time shift	100	time moves on by 10 ms to 1 s
velocity	32	loudness of the following notes, in steps of 4

Each event is one integer, 0 to 387. The sustain pedal is applied to the notes before encoding.

The fourteen events of the previous slide are the integers
`351 369 48 256 368 36 320 371 72 285 372 74 261 200`.

The whole performance becomes 8778 events, about 37 per second and 4.6 per note. Training cuts it into windows of 512 events, about 14 seconds of this piece; a batch of 64 windows is a tensor of shape (512, 64).

Timing and loudness are part of the vocabulary. The model learns not only which notes Bach wrote, but how pianists played them: rubato, chords that are not quite together, a crescendo.

Simon & Oore, “Performance RNN”, Magenta, 2017; Oore et al., *Neural Computing and Applications* 32, 2020.

Counting transitions

A tune of fourteen notes:

C C G G A A G F F E E D D C

For every note, count which note follows it. C is followed once by C and once by G; the final C is followed by nothing. G is followed by G, by A and by F.

Divided by the number of times each note was left, the counts become probabilities: a **transition table**. This is a first-order Markov chain: the next note depends only on the current one.

from → to	C	D	E	F	G	A
C	½				½	
D	½	½				
E		½	½			
F			½	½		
G				⅓	⅓	⅓
A					½	½

To generate, start from a note and draw the next from its row, again and again:

C G F E D C C G A G F F E D C …

Every step is a transition that occurs in the tune; the tune itself never occurs.

The order of a chain

In a chain of order 2 the next note depends on the last two. In the tune above, every pair of notes is followed by only one note: after G G always A, after A G always F.

Generation from order 2 therefore reproduces the tune exactly. With fourteen notes, two notes of memory are enough to copy.

Brooks, Hopkins, Neumann and Wright (1957) built chains of order 1 to 8 from hymn tunes. At low orders the results wandered without shape; at high orders they copied whole phrases of the hymns.

Between the two lies a narrow range in which the output sounds like the corpus without repeating it. Its position depends on the size of the corpus.

A larger order also needs far more data. With 128 pitches, order 1 has 128 × 128 = 16 384 possible transitions, order 3 has 268 million. Most of them never occur in a corpus, and a chain cannot continue a context it has never seen.

Brooks, Hopkins, Neumann & Wright, “An experiment in musical composition”, *IRE Transactions on Electronic Computers* EC-6, 1957.

Markov chains in Max

Object	What it does	Messages
`anal` → `prob`	built into Max. `anal` counts pairs of successive integers and outputs *previous*, *current*, *count*; `prob` stores these as a table and draws from it	order 1 only; higher orders by encoding pairs of notes as one integer
`ml.markov`	a chain of any order; states are integers or symbols	lists of observations, then `build`; `bang` generates; `order`, `states`, `dynamic`
`ml.vmm.ppm` and four other `ml.vmm` objects	variable order: for every context it uses the longest one that occurred in the training data	arguments: alphabet size (256) and maximum order (5); learns at once from its right inlet; `logeval`

A variable-order model avoids the choice between wandering and copying: where the corpus has a long matching context it continues it, elsewhere it falls back to a shorter one, as Pachet's *Continuator* (2003) does. `logeval` works the other way round: it measures how probable a given phrase is under the model.

Max 9 reference; ml.star 1.3 reference; Begleiter, El-Yaniv & Yona, *JAIR* 22, 2004.

ml.star

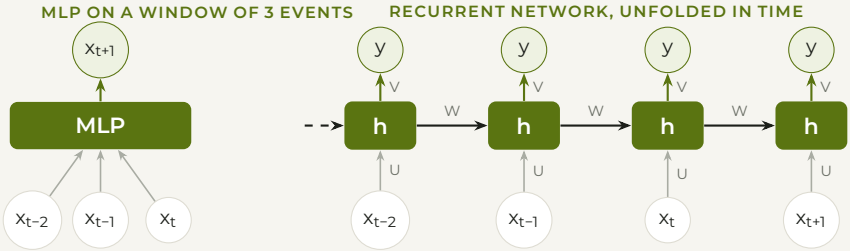
A package of seventeen objects for Max by Benjamin D. Smith (Indiana University–Purdue University Indianapolis), presented at NIME 2012, free from the Max Package Manager. Version 1.3 (2022) runs on Apple Silicon, except `ml.hmm`.

Most of its models are unsupervised and learn *online*: every new example changes the model at once, while the patch runs. There is no separate training phase, as with `fit` in FluCoMa.

`ml.markov`	Markov chain of order n
`ml.vmm.*`	variable-order Markov models
`ml.som`	self-organising map
`ml.art`	categories that grow with the data (fuzzy ART)
`ml.fcm`	fuzzy clustering
`ml.kdtree`	nearest neighbours
`ml.mlp`	multilayer perceptron
`ml.hmm`	hidden Markov model (Intel only)

Smith & Garnett, “Unsupervised play: machine learning toolkit for Max”, NIME 2012; ml.star 1.3.0 release notes.

A window or a memory



An MLP like that of Exercise 1 can predict the next event from the last three. It sees exactly three: whatever happened before is lost, as in a Markov chain of order 3.

A **recurrent network** (Elman 1990) keeps a hidden state h, a list of numbers that is passed from each step to the next. At every step it combines the new event (weights U) with its previous state (weights W), and computes from the result a prediction (weights V).

The same three sets of weights are used at every step. The network therefore has the same size for a sequence of 10 or of 10 000 events, and what it learns at one position applies at every other.

Remembering over long distances

Backpropagation through a recurrent network runs back through every step of the sequence. At each step the error is multiplied by the weights W again. Hochreiter (1991) and Bengio, Simard and Frasconi (1994) showed the consequence: the error signal shrinks, or grows, exponentially with the distance. A plain recurrent network learns relations over a few steps; it does not learn that the theme of bar 1 returns in bar 17.

LSTM, 1997

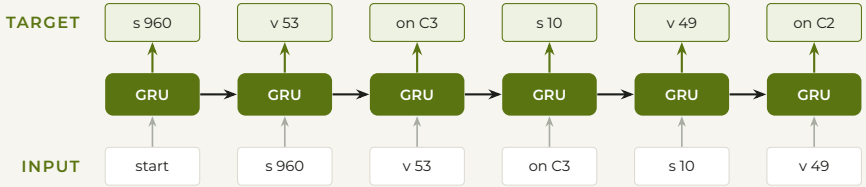
Hochreiter and Schmidhuber added a memory cell that is carried from step to step unchanged unless *gates* change it. Three gates, each a small layer with outputs between 0 and 1, decide what is written into the cell, what is erased and what is read out.

GRU, 2014

Cho et al. simplified the design to two gates and no separate cell: an *update* gate decides how much of the old state to keep, a *reset* gate how much of it to use for the new one. With fewer parameters it learns about as well. Performance RNN uses GRU layers.

Hochreiter & Schmidhuber, *Neural Computation* 9(8), 1997; Cho et al., EMNLP 2014.

Training on the next event



The target at every step is the next event of the same performance: the window shifted by one. At the output the network gives 388 probabilities, one per event, and the loss is small when the event that really follows got a high probability.

The loss is the *cross entropy*: minus the logarithm of the probability given to the right event, averaged over all steps. Its exponential is the number of equally likely events the model is, in effect, choosing among.

Reading the loss

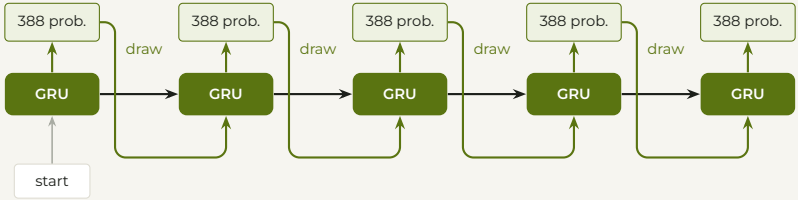
	Loss	Choosing among
a random choice among the 388 events	5.96	388
the frequency of each event, without context (Bach in MAESTRO)	4.57	97
Performance RNN, 3 LSTM layers of 1024 units (Huang et al. 2018)	1.97	7.2
Music Transformer, same data	1.84	6.3

Even the best models stay near 2, about seven equally likely choices at every step. Exact timing and loudness cannot be predicted: two pianists, or one pianist on two evenings, play the same bar differently.

Compare only losses measured on the same data and with the same window length: the first events of each window are predicted without context, so longer windows give a lower loss. When the validation loss rises while the training loss keeps falling, the model is learning its training files by heart (lesson 1, overfitting).

Huang et al. 2018, table 3; `performance_rnn_torch` README, “Reading the validation loss”.

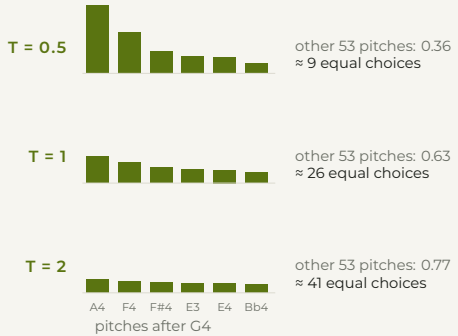
Generating: one event at a time



In training the input of each step is the true previous event. In generation there is none: the network draws an event from its own probabilities and receives it as the next input. A model that feeds on its own output in this way is *autoregressive*.

Errors therefore accumulate. A wrong event at step 100 becomes the context of every later step, and the model has never been trained on contexts that contain its own mistakes. Long generated performances drift, lose their key, or get stuck on one figure.

Temperature, with real numbers



In the 144 Bach performances of MAESTRO, G4 is followed by 59 different pitches. The six most frequent cover 37% (T = 1); the bars show their probabilities.

Temperature divides the network's scores by T before they are turned into probabilities, which raises every probability to the power 1/T. At T = 0.5 the frequent choices become more frequent: A4 from 10% to 25%. At T = 2 the distribution flattens and rare pitches gain.

The numbers beside the bars give how many equally likely choices each distribution amounts to: 9, 26, 41. Low temperature repeats figures; high temperature produces wrong notes and broken rhythms, because time shifts and velocities are drawn from flattened distributions too.

Successive note onsets in 144 Bach performances of MAESTRO v3.0.0, 362 857 notes; as a Markov chain of order 1, not a network.

Top-k and top-p

Temperature changes every probability. Two other methods cut the long tail of rare events, which at any temperature adds up to a large share: after G4, 63% of the probability lies outside the six most frequent pitches.

TOP-K

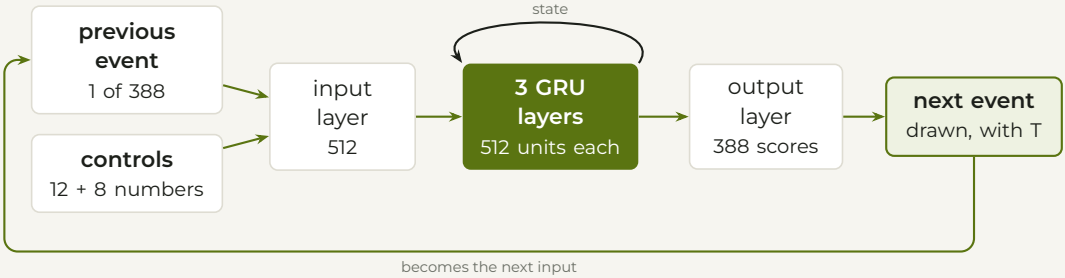
Keep only the k most probable events, divide by their sum, draw among them. With k = 1 the model always plays its most probable event: generation becomes deterministic and soon repeats itself.

TOP-P

Keep the smallest set of events whose probabilities add up to p, e.g. 0.95. Where the model is sure, few events remain; where it hesitates, many. Holtzman et al. (2020) introduced it for text as *nucleus sampling*.

In `prnn generate`: `-T` temperature, `-k` top-k, `-p` top-p. The real-time server of Exercise 3 exposes temperature only, as an OSC message.

The model



The model of `performance_rnn_torch`, with the settings of Magenta's Performance RNN (Simon and Oore, 2017). With the defaults it has **5 338 392 parameters**; the network of Exercise 1 had 86. Three quarters of them are in the GRU layers: each has 3 × (2 × 512 × 512 + 2 × 512) = 1 575 936.

Besides the previous event, every step receives a **control vector**: what the music should be like in the next seconds. During training the controls are measured from the performance itself; during generation you set them. Half of the training batches are given no controls, so that the model also plays without.

Two controls

NOTE DENSITY

Notes per second in the next 3 seconds, in 8 classes:

0	1	2	3	4	5	6	7
< 1	1–2	2–4	4–8	8–16	16–32	32–64	≥ 64

The Bach prelude above has 1925 notes in 240 seconds, 8 per second on average: class 3 or 4, depending on the passage.

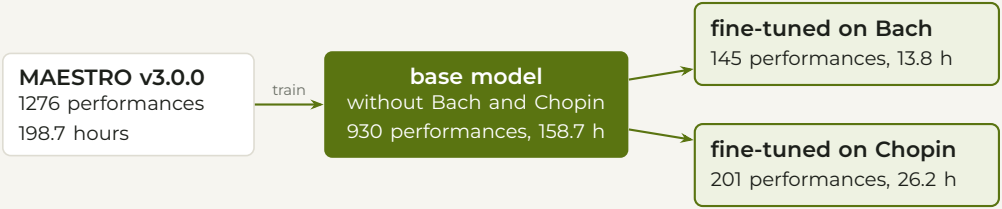
PITCH-CLASS HISTOGRAM

Twelve numbers, C to B: how long each pitch class sounds in the next 5 seconds, as a share of the total. The octave is ignored. A histogram with weight only on C, E and G asks for C major harmony; equal weights leave the choice to the model.



The controls are a request, not a rule: the model was trained to continue in a way that matches them, and follows them more or less well. The histogram returns in the second half of the lesson, where a map of the corpus produces it.

The models of Exercise 3



A hundred performances of one composer are few for five million parameters: trained on them alone, the model soon learns them by heart. Most of what it has to learn is common to all the composers: how pianists shape time and loudness, how chords lie under the hand, how notes are held and released.

The base model learns this from the rest of MAESTRO. Training then continues on one composer, starting from its weights, with a learning rate ten times lower: *fine-tuning*. Bach and Chopin are left out of the first stage so that no piece used to judge the fine-tuned model has been seen before. Transcriptions such as Bach–Busoni have composer names of their own and remain in the base data.

Counts from `maestro-v3.0.0.csv`; Hawthorne et al., ICLR 2019.

Performance RNN in Max



To the server, port 9010		From the server, port 9011	
`/play 1`, `/play 0`	start, stop	`/note_on` p v	a key is pressed
`/temperature` f	as on the previous slides	`/note_off` p	a key is released
`/density` f	notes per second	`/playing` 0/1	started or stopped
`/histogram` 12 floats	pitch classes C to B		
`/controls 1`, `0`	with or without controls		
`/reset`	start a new performance		

The server generates about half a second ahead of the clock and sends each note when its time comes; a change of the controls is heard after that delay. On a laptop processor it generates more than ten times faster than a performance needs.

`performance_rnn_torch` README; example patch `examples/max/prnn-serve.maxpat`.

Recurrence and attention

RECURRENT NETWORK

Everything the model knows about the past is in its state: 3 × 512 numbers after event 8000 as after event 8. What happened in bar 1 must survive thousands of updates to influence bar 17.

TRANSFORMER

Attention (lesson 1) compares every event with every earlier one and takes from each what is relevant. Bar 17 can look back at bar 1 directly. The cost grows with the square of the length of the context.

The Music Transformer (Huang et al., 2018) used the event vocabulary of Performance RNN and added *relative* attention: the model learns how much an event matters at a distance of one beat, or of one bar, wherever it occurs. On the same piano data its loss is 1.84 against 1.97, and in a listening test its continuations were preferred to those of Performance RNN.

Performance RNN is used here because it is small enough to train on a laptop or a gaming PC, and fast enough to play in real time on a processor.

Huang et al., “Music Transformer”, arXiv:1809.04281, 2018; ICLR 2019.

Pretrained models of music, 2022–2026

Model	What it does	Data, licence
Notochord Shepardson et al., 2022	recurrent model of MIDI events for live playing: answers in under 10 ms, any instrument	Lakh MIDI; open source
Anticipatory Music Transformer Thickstun et al., 2023	generates an accompaniment for a given melody, or fills a gap between two passages	Lakh MIDI; weights public
Aria Bradshaw et al., 2025	continues a piano performance; Aria-Duet plays in alternation with a pianist on a Disklavier	Aria-MIDI, transcribed piano recordings
YuE2, 2026	writes melody and chords as an editable score (ABC notation), then the audio of a song	weights CC BY-NC 4.0

The method is the same as in Exercise 3: a corpus of events, the next event as target, sampling. What has changed is the size of corpus and model, and the role of the symbolic layer: in YuE2 it is the plan that an audio model then realises.

Shepardson, Armitage & Magnusson, AIMC 2022; Thickstun, Hall, Donahue & Liang, *TMLR* 2024; Bradshaw et al., ISMIR 2025 and arXiv:2511.01663; github.com/multimodal-art-projection/YuE.

A map that learns while you play

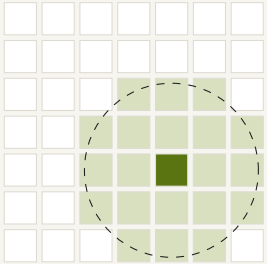
In Exercise 2 UMAP placed recorded hand shapes on a plane. It was fitted once on a finished dataset; new points could be placed on the map, but the map did not change, and a point of the plane could not be turned back into a hand shape.

Kohonen's **self-organising map** (1982) is a grid of nodes. Each node holds a list of weights with as many numbers as an example: 12 for a pitch-class histogram, 63 for a hand, n for the n parameters of a synthesizer.

Every node is therefore a prototype: a point of the map is an example, and the map can be played back directly. It learns one example at a time, without targets, and can keep learning during a performance.

Kohonen, "Self-organized formation of topologically correct feature maps", *Biological Cybernetics* 43, 1982.

How the map learns

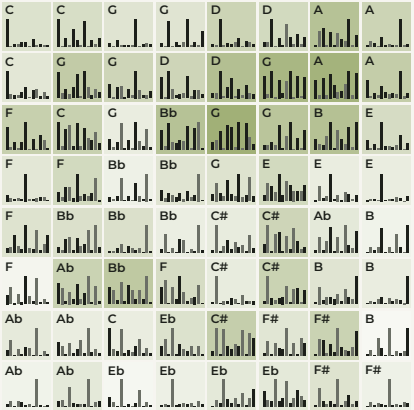


best match; its neighbours within the radius move too

1. The weights start at random values.
2. An example arrives. The node whose weights are closest to it is the *best match*.
3. The best match moves its weights part of the way towards the example: the learning rate.
4. So do its neighbours on the grid, less the further they are: the neighbourhood radius.
5. Repeat for many examples, shrinking the radius and the rate.

Because neighbours are pulled along, similar examples end up on nearby nodes: the grid unfolds over the data.

A map of Bach and Chopin



28 366 windows of 5 seconds from the Bach and Chopin performances of MAESTRO, each a pitch-class histogram as in Performance RNN. An 8 × 8 map; each cell draws its 12 weights as bars, C to B, black keys in grey, and is labelled with its strongest pitch class.

Nobody told the map about keys. Along the top row the strongest pitch class goes C, G, D, A: each step a fifth up. Down the right side it continues with E, B, F#. Keys a fifth apart lie next to each other; those with many flats gather at the bottom left.

The background is greener where more of the windows that land on a cell are Bach's. In these performances Bach is most frequent in the upper half, the keys with few accidentals; Chopin fills the rest. The map was never told who wrote what.

Computed with numpy (`tools/som_13.py`): Kohonen's rule, 60 000 steps, radius 4 to 0.25, learning rate 0.5 to 0.01.

The map as a controller

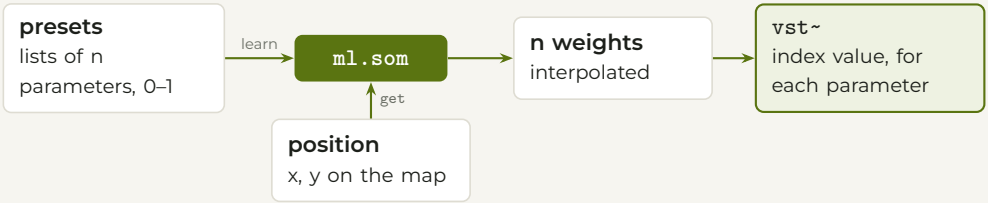


`get` with two coordinates outputs the weights of a node; with coordinates between nodes, `ml.som` interpolates between them. Moving across the map moves continuously through the harmonies of the corpus, and the model is asked to play in them.

The map can be trained on Bach and played with the Chopin model, or the reverse: the harmony comes from one corpus, the way of playing from the other.

`ml.som`	arguments: size (a square grid), neighbourhood, learning rate, plasticity, e.g. `ml.som 16 4 0.5 0.1`
`plasticity`	how quickly nodes stop changing: 0 never, 1 at once
`randomize`, `reset`	random weights; all weights to 0

A map of a plugin



A synthesizer or an effect has dozens of parameters. Each preset you save is a list of their values, 0 to 1; `vst~` reports a parameter with `get` and sets it with *index value*. The map arranges the presets on a plane and fills the space between them with intermediate settings.

The plane becomes a performance surface with two dimensions, played with the mouse, a touch on the phone of Exercise 1, or the hand map of Exercise 2. The three parameters of the FM synthesizer of Exercise 1 work the same way.

The map arranges presets by how close their parameters are, not by how they sound: a change of one parameter, a filter cutoff, can change the sound completely. Adding audio descriptors of each preset to its list makes the map take the sound into account.

Max 9 reference, `vst~`: parameters numbered from 1, values 0 to 1.

Two maps compared

	`fluid.umap~`, Exercise 2	`ml.som`, Exercise 3
Result	a position for every example	a grid of nodes, each with weights like an example
Training	on a whole dataset, with `fittransform`	one example at a time, as it arrives
New input	placed with `transformpoint`	the best-matching node; it may also change the map
From the plane back to the data	no inverse	`get` x y, with interpolation
Resolution	as many points as examples	as many nodes as the grid has, e.g. 16 × 16 = 256

UMAP is the better tool for looking at a dataset; the SOM is the better tool for playing one. `ml.art`, in the same package, takes the online idea further: it creates a new category whenever an input does not match any existing one closely enough (`vigilance`, 0.85 by default), and can be left listening to a performer.

FluCoMa 1.0.9 and ml.star 1.3 references; Carpenter, Grossberg & Rosen, *Neural Networks* 4, 1991.

Exercise 3: four patches

	Trained on	Model	You steer	Sound
A	your MIDI files	`ml.markov`, `anal` → `prob`, `ml.vmm.ppm`	order, tempo	`noteout`
B	MAESTRO, then Bach or Chopin	Performance RNN, `prnn serve`	temperature, density, pitch classes; with a sensor	`noteout`
C	histograms of MIDI files	`ml.som`, then `prnn serve`	a position on the map	`noteout`
D	presets of a plugin	`ml.som`	a position on the map	`vst~`

The patches are in `max/exercise3/`; they are independent and can be done in any order. Parts A and D run without Python. For B and C, `spawn` starts the server of `performance_rnn_torch` from the patch, as it started the tracker of Exercise 2.

EXERCISE 3

Part A: a Markov chain of your music

1. Open `ex3a-markov.maxpat`, read one of the MIDI files you brought, click *learn*: `seq` plays it 16 times faster, its note-on pitches go to `ml.markov`; at the end of the file, `build`.
2. Turn on *play*: every step generates a pitch, played by `makenote` and `noteout`. Duration and velocity are fixed.
3. Set the order to 1, 2, 3, 5, each time *clear* and *learn*. Find the order at which you start to recognise passages of your piece.
4. Do the same with `anal` → `prob`, which stops at order 1, and with `ml.vmm.ppm`, which chooses the order itself.
5. Take the rhythm from a second chain, trained on the time between onsets in six classes. Then *learn* a second piece without *clear*: both are in the chains.

Part B: Performance RNN, steered

1. Open `ex3b-prnn.maxpat`. Click the message of section 1: `spawn` starts `prnn serve` with the Bach model; the console shows *listening on port 9010*.
2. Turn on *play*. Choose a synthesizer for `noteout`, on macOS e.g. AU DLS Synth.
3. Move the temperature from 0.5 to 1.5, then the density from 2 to 20 notes per second. Listen for when the model breaks down; section 4 shows what it plays next to what you asked.
4. Set the histogram to three pitch classes, then to one. Turn the controls off (`/controls 0`) and compare.
5. Change the model to Chopin, with the same controls.
6. With `ex1-tilt-timbre.maxpat` open, *use the phone*: touch x and y set temperature and density.

Part C: a map of harmonies

1. Open `ex3c-harmony-map.maxpat`. Read the pitch-class histograms of 5-second windows of Bach and Chopin (MAESTRO), or compute new ones: `spawn` runs `histograms.py` on a folder of MIDI files, or on one composer, and the script sends them over OSC.
2. Click *train*: each node starts as a random histogram, then 40 rounds of 1500 histograms in random order, while the neighbourhood and the learning rate decrease. Each cell shows its 12 weights.
3. Start the server as in part B and turn on *play*. Click and drag on the map: the 12 weights of that point are sent as `/histogram`.
4. Train the map on your own MIDI files and play it with the Bach model, then with the Chopin model.
5. Drive the position with the phone: touch x and y are the x and y of the map.

Part D: a map of a plugin

1. Open `ex3d-plugin-map.maxpat` and load a plugin into `vst~`, a synthesizer or an effect. Choose eight to sixteen of its parameters; the patch reads their names with `params`.
2. Set the plugin by hand to a sound you like and click *add preset*: the patch reads the chosen parameters with `get` and stores them as one list. Make ten to twenty presets, as different as possible.
3. Click *train*. Then *play*: the position on the map gives, with `get` x y, a value for each parameter.
4. Play the map with the mouse, then with a sensor. Find the regions between your presets: settings you never made.
5. Without a plugin of your own: `AUDistortion`, included in macOS, on the drum loop of the patch.

EXERCISE 3

Variations

Change	Observe
A: train on two pieces in different keys	where the chain moves from one key to the other
A: `ml.vmm.ppm`, `logeval` of phrases of another piece	which phrases the model finds probable
B: temperature 0.2 and 2.0	repetition; loss of rhythm
B: histogram from part C, density from Exercise 1	two controllers for one model
C: map 4 × 4, then 32 × 32	a few harmonies, or many fine steps
D: `plasticity 0`, keep adding presets while playing	a map that changes during the performance
D: presets with audio descriptors added	a map ordered by sound

EXERCISE 3

Pd and Plugdata

Part A: `cyclone/anal` and `cyclone/prob`, included in Plugdata, behave like the Max objects; higher orders by encoding two pitches as one number, e.g. 128 × previous + current.

Part B: `examples/pd/prnn-serve.pd` in the repository starts the server with `spawn` and plays the notes with a small built-in synthesizer.

ml.star exists only for Max, so parts C and D have no Pd version with a SOM. A substitute with FluCoMa for Pd: `fluid.umap~` arranges the presets, `fluid.kdtree~` finds the preset nearest to the cursor. It jumps between presets and does not interpolate.

Training your own model

Once, with uv installed:

```
git clone https://github.com/anatrini/performance_rnn_torch && cd performance_rnn_torch && uv sync --all-extras
```

1 · BASE MODEL, WITHOUT THE COMPOSERS YOU WILL FINE-TUNE ON

```
uv run prnn prepare --all -x "Johann Sebastian Bach" -x "Frédéric Chopin"
uv run prnn preprocess --midi-root data/midi-maestro --save-dir data/processed-maestro
uv run prnn train -S models/maestro.sess -d data/processed-maestro -r 0.1
```

2 · FINE-TUNING ON ONE COMPOSER

```
uv run prnn prepare --composer "Frédéric Chopin"
uv run prnn preprocess
uv run prnn train -S models/chopin-ft.sess --init-from models/maestro.sess -l 1e-4 -M "gru_dropout=0.3"
```

Windows with an NVIDIA card. The default PyTorch package does not use the card. After `uv sync`: `uv pip install --reinstall torch --torch-backend=auto`, then `uv run --no-sync prnn train …`

Time. On an M1 Pro an epoch of MAESTRO without Bach takes about an hour and a half. Training stops when the validation loss has not improved for 5 epochs.

Your own files go in `data/midi/<name>/`; piano music works best. Files shorter than a window of 512 events are not used: lower `-w`.

`performance_rnn_torch` README: installation, quick start, fine-tuning.

MIDI datasets

Dataset	Content	Licence
MAESTRO v3.0.0	1276 piano performances recorded on Disklaviers, 199 hours, classical repertoire	CC BY-NC-SA 4.0
GiantMIDI-Piano	10 855 piano works by 2786 composers, transcribed from recordings	CC BY 4.0
Aria-MIDI	about 1.19 million files, about 100 000 hours of solo piano, transcribed, with metadata on composer and genre	CC BY-NC-SA 4.0
PiJAMA	solo jazz piano: 2777 performances, 223 hours, 120 pianists, transcribed	see the project page
Lakh MIDI	176 581 files of all genres and instruments, collected from the web	CC BY 4.0
PDMX	about 250 000 scores from MuseScore marked public domain, MusicXML	public domain; use the subset without licence conflicts
Mutopia Project	2124 engraved scores with MIDI, mostly before 1900	public domain or CC, stated per piece

Transcribed data contain the errors of the transcription model; files collected from the web contain sequenced, not played, music: quantised time and constant velocities. Both change what a model learns.

Data, model, training in Exercise 3

Data

your MIDI files; 199 hours played by competitors of the Piano-e-Competition; your presets

Model

a table of counts, a recurrent network of five million parameters, a grid of prototypes

Training

which composers were left out, the order of the chain, the size of the map

The three models share one property: they produce only what is probable under their data. A chain cannot play a transition it has never counted; the network gives a probability to everything, but little to what MAESTRO does not contain; the map interpolates between the examples it has seen.

In Audry's terms the creative decisions move from the notes to the corpus and the controls: what goes into the data, and what you steer while the model plays.

Audry 2021, chapters on the model.

Whose performances

MAESTRO was recorded at the International Piano-e-Competition, in Minneapolis, on Yamaha Disklaviers. The pianists played for a competition, not for a dataset. Google's Magenta team released the recordings in partnership with the competition, under a licence (CC BY-NC-SA 4.0) that forbids commercial use and requires adaptations to be shared under the same terms.

A model trained on it is made from these performances. The licence was written for the data; it does not say whether a trained model, or music generated with it, counts as an adaptation. The README of `performance_rnn_torch` asks you to keep this in mind for models trained on MAESTRO.

Copying can be checked. A chain of high order, or a model fine-tuned on few pieces, may reproduce passages of its corpus note for note. Comparing the generated events with the training files, for instance by searching the longest shared sequence, shows how much of a generated performance is new.

Hawthorne et al., "Enabling factorized piano music modeling and generation with the MAESTRO dataset", ICLR 2019.

Questions

1. In part A, at which order did the chain start to copy your piece? What does that say about the amount of music in your corpus?
2. The fine-tuned model plays “like Bach” with the timing of contemporary pianists. Whose style is it?
3. A map of your presets produces settings you never made. Are they yours?

DISCUSSION

Before the next block

- Read Audry, *Art in the Age of Machine Learning*, the part on the model.
- Finish one part of Exercise 3 and record one minute of playing it.
- If you have a computer with an NVIDIA card: start training a base model without the composers you want, and fine-tune it before block 4.
- Block 4 generates sound, not notes. Bring two or three recordings of a single instrument or voice, a few minutes each.

References

Audry, S. (2021). *Art in the Age of Machine Learning*. MIT Press.
Begleiter, R., El-Yaniv, R., & Yona, G. (2004). On prediction using variable order Markov models. *Journal of Artificial Intelligence Research*, 22.
Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2).
Bradshaw, L., & Colton, S. (2025). Aria-MIDI: a dataset of piano MIDI files for symbolic music modeling. *ICLR*.
Brooks, F. P., Hopkins, A. L., Neumann, P. G., & Wright, W. V. (1957). An experiment in musical composition. *IRE Transactions on Electronic Computers*, EC-6(3).
Carpenter, G. A., Grossberg, S., & Rosen, D. B. (1991). Fuzzy ART. *Neural Networks*, 4.
Cho, K., et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*.
Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2).
Hawthorne, C., et al. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. *ICLR*.
Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8).
Holtzman, A., et al. (2020). The curious case of neural text degeneration. *ICLR*.
Huang, C.-Z. A., et al. (2018). Music Transformer. arXiv:1809.04281; *ICLR* 2019.
Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43.
Oore, S., Simon, I., Dieleman, S., Eck, D., & Simonyan, K. (2020). This time with feeling. *Neural Computing and Applications*, 32.
Pachet, F. (2003). The Continuator: musical interaction with style. *Journal of New Music Research*, 32(3).
Shepardson, V., Armitage, J., & Magnusson, T. (2022). Notochord: a flexible probabilistic model for real-time MIDI performance. *AIMC*.
Smith, B. D., & Garnett, G. E. (2012). Unsupervised play: machine learning toolkit for Max. *NIME*.
Thickstun, J., Hall, D., Donahue, C., & Liang, P. (2024). Anticipatory Music Transformer. *TMLR*.

performance_rnn_torch **spawn** **MAESTRO** **Aria**